

Items to rate (36 MRs) – readable form

Each MR runs the program twice. JIR = how the two *inputs* relate; JOR = the *output* relation that must hold. Subscripts 1, 2 = first/second call; **re/im** = real/imag part; **this/other** = operands; **out** = return. Classify each into one MR family (a-j) or **orphan** (see CODEBOOK).

M01 ComplexSignal.add.0 / G_swap_negate

Program: add(other): complex addition; out = this + other (real and imaginary parts added separately). Operands this=(this.re,this.im), other=(other.re,other.im); result out=(out.re,out.im).

JIR: $(\text{this.re}_1 \approx \text{other.re}_2) \wedge (\text{this.im}_1 \approx \text{other.im}_2) \wedge (\text{other.re}_1 \approx \text{this.re}_2) \wedge (\text{other.im}_1 \approx \text{this.im}_2)$

JOR: $\Rightarrow (\text{out.re}_1 \approx \text{out.re}_2) \wedge (\text{out.im}_1 \approx \text{out.im}_2)$

M02 ComplexSignal.add.0 / L_scale

Program: add(other): complex addition; out = this + other (real and imaginary parts added separately). Operands this=(this.re,this.im), other=(other.re,other.im); result out=(out.re,out.im).

JIR: $(\text{this.re}_1 \approx 2.0 \cdot \text{this.re}_2) \wedge (\text{this.im}_1 \approx 2.0 \cdot \text{this.im}_2) \wedge (\text{other.re}_1 \approx 2.0 \cdot \text{other.re}_2) \wedge (\text{other.im}_1 \approx 2.0 \cdot \text{other.im}_2)$

JOR: $\Rightarrow (\text{out.re}_1 \approx 2.0 \cdot \text{out.re}_2) \wedge (\text{out.im}_1 \approx 2.0 \cdot \text{out.im}_2)$

M03 ComplexSignal.add.0 / T_shift_re

Program: add(other): complex addition; out = this + other (real and imaginary parts added separately). Operands this=(this.re,this.im), other=(other.re,other.im); result out=(out.re,out.im).

JIR: $((\text{this.re}_1 \approx (\text{this.re}_2 + 1.0)) \wedge (\text{this.im}_1 \approx \text{this.im}_2) \wedge (\text{other.re}_1 \approx \text{other.re}_2) \wedge (\text{other.im}_1 \approx \text{other.im}_2)$

JOR: $\Rightarrow ((\text{out.re}_1 \approx (\text{out.re}_2 + 1.0)) \wedge (\text{out.im}_1 \approx \text{out.im}_2))$

M04 MathSignalClass.clamp.0 / G_negate_bound_swap

Program: clamp(lo, hi, x): clamp x into the interval [lo, hi]; returns min(max(x, lo), hi).

JIR: $((lo_1 \approx (-hi_2)) \wedge ((hi_1 \approx (-lo_2)) \wedge ((x_1 \approx (-x_2))))$

JOR: $\Rightarrow out_1 \approx (-out_2)$

M05 MathSignalClass.clamp.0 / L_scale

Program: clamp(lo, hi, x): clamp x into the interval [lo, hi]; returns min(max(x, lo), hi).

JIR: $(lo_1 \approx 2.0 \cdot lo_2) \wedge (hi_1 \approx 2.0 \cdot hi_2) \wedge (x_1 \approx 2.0 \cdot x_2)$

JOR: $\Rightarrow out_1 \approx 2.0 \cdot out_2$

M06 MathSignalClass.clamp.0 / 0_le_mono_in_x

Program: clamp(lo, hi, x): clamp x into the interval [lo, hi]; returns min(max(x, lo), hi).

JIR: $(lo_2 \approx lo_1) \wedge (hi_2 \approx hi_1) \wedge (x_2 \leq x_1)$

JOR: $\Rightarrow out_2 \leq out_1$

M07 MathSignalClass.clamp.0 / T_shift

Program: clamp(lo, hi, x): clamp x into the interval [lo, hi]; returns min(max(x, lo), hi).

JIR: $((lo_1 \approx (lo_2 + 1.0)) \wedge ((hi_1 \approx (hi_2 + 1.0)) \wedge ((x_1 \approx (x_2 + 1.0))))$

JOR: $\Rightarrow out_1 \approx (out_2 + 1.0)$

M08 MathSignalClass.exactLog2.0 / L_idem_at_one

Program: exactLog2(n): base-2 logarithm of a positive integer n (so exactLog2(2*n) = exactLog2(n) + 1).

JIR: $n_1 = 2 \cdot n_2$

JOR: $\Rightarrow out_1 = (out_2 + 1)$

M09 MathSignalClass.exactLog2.0 / 0_le_monotone

Program: exactLog2(n): base-2 logarithm of a positive integer n (so exactLog2(2*n) = exactLog2(n) + 1).

JIR: $n_2 > 0 \wedge (n_2(n_2 - 1)) = 0 \wedge n_1 > 0 \wedge (n_1(n_1 - 1)) = 0 \wedge n_2 \leq n_1$

JOR: $\Rightarrow out_2 \leq out_1$

M10 MathSignalClass.exactLog2.0 / T_double

Program: exactLog2(n): base-2 logarithm of a positive integer n (so exactLog2(2*n) = exactLog2(n) + 1).

JIR: $n_1 \approx 2.0 \cdot n_2$

JOR: $\Rightarrow out_1 \approx (out_2 + 1.0)$

M11 MathSignalClass.gcdSig.0 / G_swap_negate

Program: gcdSig(a, b): greatest common divisor of |a| and |b| (inputs sign-normalised, then Euclidean gcd).

JIR: $(a_1 = b_2) \wedge (b_1 = a_2)$

JOR: $\Rightarrow out_1 = out_2$

M12 MathSignalClass.gcdSig.0 / L_scale

Program: gcdSig(a, b): greatest common divisor of |a| and |b| (inputs sign-normalised, then Euclidean gcd).

JIR: $a_1 = 2L \cdot a_2 \wedge b_1 = 2L \cdot b_2$

JOR: $\Rightarrow out_1 = 2L \cdot out_2$

M13 MathSignalClass.hypotSig.0 / G_swap_negate

Program: hypotSig(a, b): Euclidean hypotenuse, $\sqrt{a^2 + b^2}$.

JIR: $(a_1 \approx b_2) \wedge (b_1 \approx a_2)$

JOR: $\Rightarrow out_1 \approx out_2$

- M14** `MathSignalClass.hypotSig.0 / L_scale`
Program: `hypotSig(a, b)`: Euclidean hypotenuse, $\sqrt{a^2 + b^2}$.
JIR: $(a_1 \approx 2.0 \cdot a_2) \wedge (b_1 \approx 2.0 \cdot b_2)$
JOR: $\Rightarrow \text{out}_1 \approx 2.0 \cdot \text{out}_2$
- M15** `MathSignalClass.hypotSig.0 / 0_le_mono_in_abs`
Program: `hypotSig(a, b)`: Euclidean hypotenuse, $\sqrt{a^2 + b^2}$.
JIR: $(|a_2| \leq |a_1|) \wedge (b_2 \approx b_1)$
JOR: $\Rightarrow \text{out}_2 \leq \text{out}_1$
- M16** `MathSignalClass.isSequence.0 / B_rel_xor_reverse`
Program: `isSequence(a, b, c)`: returns true iff a, b, c form an arithmetic sequence ($b - a == c - b$).
JIR: $(a_1 \approx c_2) \wedge (b_1 \approx b_2) \wedge (c_1 \approx a_2)$
JOR: $\Rightarrow \neg(\text{out}_2 \wedge \text{out}_1)$
- M17** `MathSignalClass.isSequence.0 / G_negate_reverse`
Program: `isSequence(a, b, c)`: returns true iff a, b, c form an arithmetic sequence ($b - a == c - b$).
JIR: $((a_1 \approx (-c_2))) \wedge ((b_1 \approx (-b_2))) \wedge ((c_1 \approx (-a_2)))$
JOR: $\Rightarrow \text{out}_1 = \text{out}_2$
- M18** `MathSignalClass.isSequence.0 / L_scale_pos`
Program: `isSequence(a, b, c)`: returns true iff a, b, c form an arithmetic sequence ($b - a == c - b$).
JIR: $(a_1 \approx 2.0 \cdot a_2) \wedge (b_1 \approx 2.0 \cdot b_2) \wedge (c_1 \approx 2.0 \cdot c_2)$
JOR: $\Rightarrow \text{out}_1 = \text{out}_2$
- M19** `MathSignalClass.isSequence.0 / 0_le_transitivity`
Program: `isSequence(a, b, c)`: returns true iff a, b, c form an arithmetic sequence ($b - a == c - b$).
JIR: $(a_1 \leq a_2) \wedge (b_2 \leq b_1) \wedge (c_2 \leq c_1)$
JOR: $\Rightarrow (\neg \text{out}_2) \vee \text{out}_1$
- M20** `MathSignalClass.isSequence.0 / T_rev_exclusion`
Program: `isSequence(a, b, c)`: returns true iff a, b, c form an arithmetic sequence ($b - a == c - b$).
JIR: $(a_1 \approx c_2) \wedge (b_1 \approx b_2) \wedge (c_1 \approx a_2)$
JOR: $\Rightarrow \neg(\text{out}_2 \wedge \text{out}_1)$
- M21** `MathSignalClass.isSequence.0 / T_shift`
Program: `isSequence(a, b, c)`: returns true iff a, b, c form an arithmetic sequence ($b - a == c - b$).
JIR: $((|(a_1 - a_2) - (b_1 - b_2)| < \varepsilon)) \wedge ((|(b_1 - b_2) - (c_1 - c_2)| < \varepsilon))$
JOR: $\Rightarrow \text{out}_1 = \text{out}_2$
- M22** `MathSignalClass.lcmSig.0 / G_swap_negate`
Program: `lcmSig(a, b)`: least common multiple of $|a|$ and $|b|$ (inputs sign-normalised).
JIR: $(a_1 = b_2) \wedge (b_1 = a_2)$
JOR: $\Rightarrow \text{out}_1 = \text{out}_2$
- M23** `MathSignalClass.lcmSig.0 / L_scale`
Program: `lcmSig(a, b)`: least common multiple of $|a|$ and $|b|$ (inputs sign-normalised).
JIR: $(a_1 = 2 \cdot a_2) \wedge (b_1 = 2 \cdot b_2)$
JOR: $\Rightarrow \text{out}_1 = 2 \cdot \text{out}_2$
- M24** `MathSignalClass.midpoint.0 / G_swap_negate`
Program: `midpoint(a, b)`: arithmetic midpoint, $(a + b) / 2$.
JIR: $(a_1 \approx b_2) \wedge (b_1 \approx a_2)$
JOR: $\Rightarrow \text{out}_1 \approx \text{out}_2$
- M25** `MathSignalClass.midpoint.0 / L_scale`
Program: `midpoint(a, b)`: arithmetic midpoint, $(a + b) / 2$.
JIR: $(a_1 \approx 2.0 \cdot a_2) \wedge (b_1 \approx 2.0 \cdot b_2)$
JOR: $\Rightarrow \text{out}_1 \approx 2.0 \cdot \text{out}_2$
- M26** `MathSignalClass.midpoint.0 / 0_le_mono_in_a`
Program: `midpoint(a, b)`: arithmetic midpoint, $(a + b) / 2$.
JIR: $a_2 \leq a_1 \wedge (b_2 \approx b_1)$
JOR: $\Rightarrow \text{out}_2 \leq \text{out}_1$
- M27** `MathSignalClass.midpoint.0 / T_shift`
Program: `midpoint(a, b)`: arithmetic midpoint, $(a + b) / 2$.
JIR: $((a_1 \approx (a_2 + 1.0))) \wedge ((b_1 \approx (b_2 + 1.0)))$
JOR: $\Rightarrow \text{out}_1 \approx (\text{out}_2 + 1.0)$
- M28** `MathSignalClass.powerSig.0 / D_reciprocal_exp`
Program: `powerSig(base, n)`: $|base|$ raised to the power n (base sign-normalised before exponentiation).
JIR: $(base_1 \approx base_2) \wedge base_2 \neq 0.0 \wedge n_1 = -n_2$
JOR: $\Rightarrow \text{out}_1 \approx (1.0/\text{out}_2)$

M29 MathSignalClass.powerSig.0 / E_power_of_power

Program: powerSig(base, n): |base| raised to the power n (base sign-normalised before exponentiation).

JIR: $base_1 \approx out_2$

JOR: $\Rightarrow out_1 \approx powerSig(base_2, n_2 \cdot n_1)$

M30 MathSignalClass.powerSig.0 / G_odd_negate

Program: powerSig(base, n): |base| raised to the power n (base sign-normalised before exponentiation).

JIR: $(n_1 = n_2) \wedge ((base_1 \approx (-base_2)))$

JOR: $\Rightarrow ((n_2 \mathbf{M31} \text{ MathSignalClass.powerSig.0 / L_scale_base}$

Program: powerSig(base, n): |base| raised to the power n (base sign-normalised before exponentiation).

JIR: $(base_1 \approx 3.0 \cdot base_2) \wedge (n_1 = n_2)$

JOR: $\Rightarrow out_1 \approx (powerSig(3.0, n_2) \cdot out_2)$

M32 MathSignalClass.powerSig.0 / T_exp_step

Program: powerSig(base, n): |base| raised to the power n (base sign-normalised before exponentiation).

JIR: $base_1 = base_2 \wedge (n_1 = (n_2 + 1))$

JOR: $\Rightarrow out_1 \approx (out_2 \cdot base_2)$

M33 MathSignalClass.signum.0 / D_reciprocal

Program: signum(x): sign of x; returns -1, 0, or +1 for $x < 0$, $x = 0$, $x > 0$.

JIR: $x_2 \neq 0.0 \wedge (x_1 \approx (1.0/x_2))$

JOR: $\Rightarrow out_1 = out_2$

M34 MathSignalClass.signum.0 / G_negate

Program: signum(x): sign of x; returns -1, 0, or +1 for $x < 0$, $x = 0$, $x > 0$.

JIR: $x_1 \approx (-x_2)$

JOR: $\Rightarrow out_1 = -out_2$

M35 MathSignalClass.signum.0 / L_scale_pos

Program: signum(x): sign of x; returns -1, 0, or +1 for $x < 0$, $x = 0$, $x > 0$.

JIR: $x_1 \approx 3.0 \cdot x_2$

JOR: $\Rightarrow out_1 = out_2$

M36 MathSignalClass.signum.0 / O_le_monotone

Program: signum(x): sign of x; returns -1, 0, or +1 for $x < 0$, $x = 0$, $x > 0$.

JIR: $x_2 \leq x_1$

JOR: $\Rightarrow out_2 \leq out_1$